Лабораторная работа №1 Измерение времени работы подпрограммы

Вариант 1. Вычисление числа π по ряду Грегори–Лейбница

Выполнил: Александров Ярослав.

Группа 25212

Год: 2026

1. Цель лабораторной работы

Изучить основные функции оптимизирующего компилятора GCC, рассмотреть уровни оптимизации и исследовать влияние параметров оптимизации GCC на время выполнения программы.

В соответствии с методическими указаниями в работе требуется скомпилировать программу с уровнями -O0, -O1, -O2, -O3, -Os, -Ofast, -Og и для нескольких значений N измерить время ее выполнения.

2. Вариант задания

$$\pi = 4\sum_{n=0}^{\infty}\frac{(-1)^n}{2n+1} = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \ldots + 4\frac{(-1)^n}{(2n+1)} + \ldots$$

В работе реализовано вычисление числа π по ряду Лейбница:

В программе параметр N определяет количество итераций суммирования. Для четных i к сумме прибавляется очередной член ряда, для нечетных — вычитается.

3. Методика проведения работы

Для измерения времени используется высокоточный счетчик производительности Windows QueryPerformanceCounter. Сначала определяется частота счетчика QueryPerformanceFrequency, затем фиксируются начальное и конечное значения счетчика относительно выполнения функции sumc(n). Время вычисляется по формуле:

T = (end.QuadPart − start.QuadPart) / freq.QuadPart.

В качестве значений N использованы 10 000 000 000, 15 000 000 000 и 20 000 000 000. Для стандартной компиляции без явного указания уровня оптимизации получено 46,866 с при N = 15 000 000 000.

4. Листинг программы

```
long double sumc(long long n){
    long double pi = 0;

    for (long long i = 0; i <= n; i++){
        if(i % 2 == 0)
            pi += 4*((long double)(1.0/(2.0*i+1)));
        else
            pi -= 4*((long double)(1.0/(2.0*i+1)));
    }
    return pi;
}

int main(){
    long long n;
    std::cin >> n;
    //freq количество тиков в секунду
    LARGE_INTEGER freq, start, end;

    // 1. Получаем частоту счётчика
    if (!QueryPerformanceFrequency(&freq)) {
        std::cerr << "QueryPerformanceFrequency failed\n";
        return 1;
    }
    // 2. Снимаем начальное показание
    QueryPerformanceCounter(&start);

    long double pi = sumc(n);

    QueryPerformanceCounter(&end);

    double full_time = double(end.QuadPart - start.QuadPart) / double(freq.QuadPart);
    long double tick = 1.0/ (long double)freq.QuadPart;//длительность одного тика секунд  (абсолютная погрешность таймера)
    long double otn_pog = tick/full_time  * 100; // относительная погрешность в %

    std::cout << std::setprecision(15) << pi << '\n' << full_time;

    return 0;
}
```

5. Компиляция и запуск программы

Базовая команда компиляции, использованная для запуска программы:

```
g++ lab2.cpp -o main.exe
```

Запуск программы:

```
.\main.exe
```

После запуска в консоль вводится значение N. Для исследования уровней оптимизации используются, например, следующие команды:

```
g++ -O0 lab2.cpp -o main.exe

g++ -O1 lab2.cpp -o main.exe

g++ -O2 lab2.cpp -o main.exe

g++ -O3 lab2.cpp -o main.exe

g++ -Os lab2.cpp -o main.exe

g++ -Ofast lab2.cpp -o main.exe

g++ -Og lab2.cpp -o main.exe
```

6. Результаты измерений

Полученные средние значения времени выполнения представлены в таблице 1.

Уровень	N = 10 млрд, с	N = 15 млрд, с	N = 20 млрд, с
-O0	30.346	47.428	62.419
-O1	9.496	14.028	22.883

-O2	9.596	15.228	18.723
-O3	9.137	13.578	19.113
-Os	21.837	33.238	44.983
-Ofast	9.337	14.008	21.703
-Og	26.017	40.092	54.843

Таблица 1 — Среднее время выполнения программы для разных уровней оптимизации.

Для стандартной команды «g++ lab2.cpp -o main.exe» при N = 15 000 000 000 получено среднее время 46.866 с. Отсутствие ключа -O соответствует режиму без явно заданной оптимизации, то есть базовому уровню O0.

График зависимости времени выполнения от N представлен на рисунке 1.

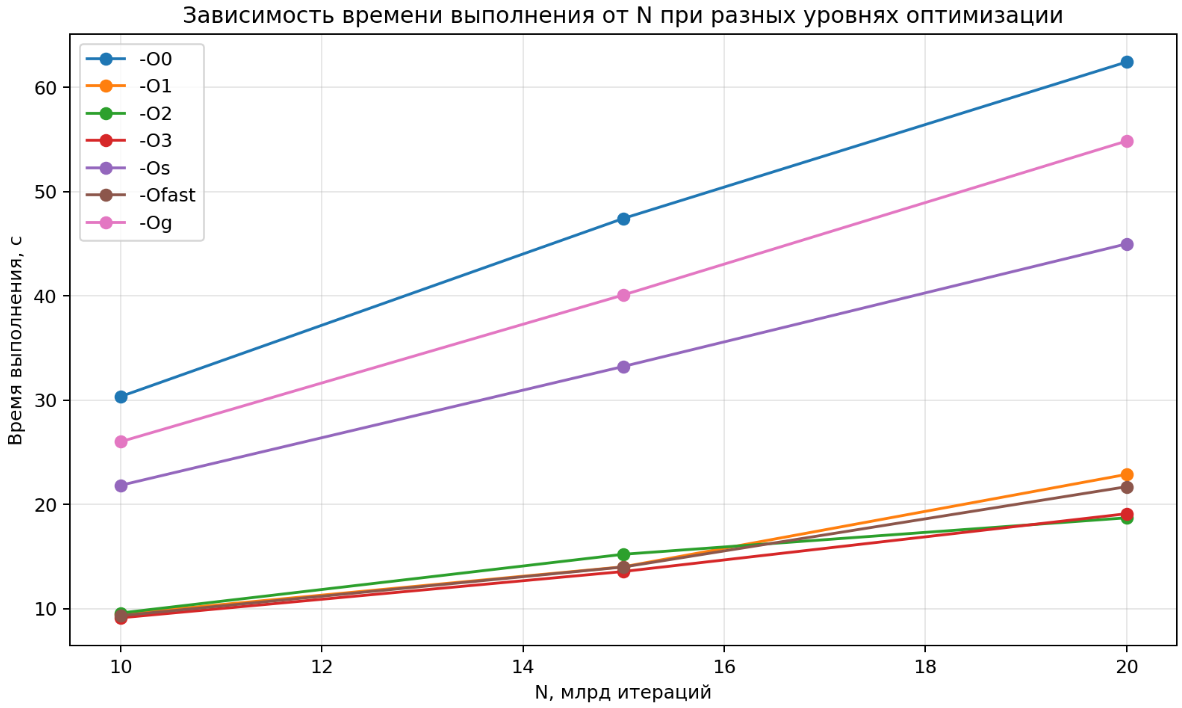


Рисунок 1 — Зависимость времени выполнения программы от N.

7. Анализ результатов

Из полученных измерений видно, что время выполнения возрастает при увеличении N. Это соответствует характеру алгоритма: количество итераций цикла непосредственно зависит от N, поэтому вычислительная нагрузка увеличивается вместе с параметром N.

Применение оптимизации существенно изменяет время выполнения относительно режима O0. Для N = 15 млрд время в режиме O0 составило 47,428 с, тогда как для O1 — 14,028 с, O2 — 15,228 с, O3 — 13,578 с, Os — 33,238 с, Ofast — 14,008 с, Og — 40,092 с.

Результаты также показывают, что увеличение номера уровня оптимизации не означает обязательного монотонного уменьшения времени выполнения. Например, на отдельных значениях N уровни O1, O2, O3 и Ofast дают близкие результаты, а уровень Os в данном эксперименте сохраняет заметно большее время выполнения. Это согласуется с методическими указаниями: конкретный эффект оптимизации определяется набором примененных преобразований и особенностями программы и процессора.

8. Вывод

В ходе лабораторной работы была исследована работа оптимизирующего компилятора GCC на примере программы вычисления числа π по ряду Лейбница. Программа была запущена для трех значений параметра N и скомпилирована с уровнями оптимизации O0, O1, O2, O3, Os, Ofast и Og. По результатам измерений установлено, что параметры оптимизации оказывают существенное влияние на время выполнения программы. При увеличении N время выполнения возрастает, а различные уровни GCC дают различный эффект. Следовательно, выбор уровня оптимизации целесообразно выполнять на основании реальных измерений времени работы конкретной программы.